

ign1t10n: macOS Installer Specification

One install: a single-validator F1R3Node-Rust shard, its observer, and the F1R3Gaze browser

Version 0.2 (draft for review)

Lucius Gregory Meredith

CEO, F1R3FLY.io, and Chief Science Officer, F1R3FLY Industries

30 September 2026

Abstract

This specification defines an installer for macOS that leaves a person, a few minutes after double-clicking one file, with three things running on their own machine: a single-validator F1R3Node-Rust shard, a read-only observer node joined to it, and the F1R3Gaze browser configured to deploy to the one and read from the other, paying from a wallet that the shard's genesis has funded. It fixes the artifact (a signed, notarised product archive, ign1t10n, carrying F1R3Gaze and the ign1t10n menu-bar application, which owns the shard), the first-run provisioning sequence, the supervision of the two nodes by a per-user launch agent, the configuration written to each program, the security model, upgrade, reset and uninstall, the build and release pipeline, and acceptance tests. It is grounded in a review of F1R3Node-Rust at [fce422a](#), F1R3Gaze at [f6ee26a](#) and CampF1R3 at [b46e16c](#) (with F1R3Gaze's pinned CampF1R3 revision [ea2d323](#)), and of the F1R3Gaze architecture proposal and implementation specification in [publications/f1r3gaze](#). The review produced eleven findings; four of them require small changes in F1R3Node-Rust and three in F1R3Gaze, specified here as work packages.

Contents

1	Scope and conventions	4
1.1	What the installer delivers	4
1.2	Non-goals for this version	4
1.3	Pinned baselines	4
1.4	Conventions	4
2	Findings from the code review	5
3	Architecture	6
3.1	Components	6
3.2	Principles	7
4	The distribution artifact	7
5	ign1t10n	8
5.1	Bundle layout	8
5.2	One binary, three roles	9

5.3	The launch agent	9
5.4	The state directory	9
5.5	Keychain items	10
6	Provisioning	10
6.1	S0: preflight	10
6.2	S1: ports	10
6.3	S2: the paying wallet	11
6.4	S3: the validator key	11
6.5	S4: genesis files	11
6.6	S5: node configurations	11
6.7	S6: register the agent	12
6.8	S7: first start	12
6.9	S8: configure F1R3Gaze	12
6.10	S9: open the browser	12
6.11	Time budget	12
7	Supervision	13
7.1	Start	13
7.2	Health	13
7.3	Restarts	13
7.4	Stop	13
7.5	Port conflicts at start	14
7.6	Sleep and wake	14
8	User interface	14
8.1	Menu bar	14
8.2	First-run window	14
9	Integration with F1R3Gaze	15
9.1	What the browser sees	15
9.2	Required changes to F1R3Gaze	15
10	Required changes to F1R3Node-Rust	15
11	Security model	16
12	Upgrade, reset and uninstall	16
12.1	Upgrade	16
12.2	Reset	16
12.3	Uninstall	17
13	Build and release pipeline	17
13.1	Where it lives	17
13.2	Jobs	17
13.3	Signing order	17

14 Testing and acceptance	18
14.1 Acceptance criteria	18
14.2 Matrix	19
15 Work packages and schedule	19
16 Obligations and decisions	19
A Configuration templates	20
B The manifest	22

1 Scope and conventions

1.1 What the installer delivers

After a successful install on a supported Mac, with no further action beyond approving a background item in System Settings when macOS asks:

1. `/Applications/F1R3Gaze.app` and `/Applications/ign1t10n.app` are installed, signed and notarised.
2. A per-user launch agent, `io.f1r3fly.ign1t10n.supervisor`, is registered and running. It supervises two `f1r3node` processes: a *validator* in standalone mode that created the shard's genesis, and an *observer* (read-only node) that joined it over loopback.
3. The validator's key was generated on this machine at install time. The shard's genesis funds the user's active F1R3Gaze wallet.
4. F1R3Gaze is configured with the validator as its deploy target and the observer as its only observer, and is open in a window.
5. Every listening socket of both nodes is bound to the loopback interface.
6. The shard survives logout, sleep and reboot, and comes back with its chain intact.

1.2 Non-goals for this version

Multi-validator local shards; joining a remote or public shard; exposing the local shard to the network; Linux and Windows (the F1R3Gaze release already ships those installers, but the shard bundle here is macOS-only); bundling the Embers service; Docker. Each is compatible with the architecture below and is noted where it would enter.

1.3 Pinned baselines

Source	Revision	Notes
F1R3Node-Rust	master fce422a	@ crate node v0.4.46; toolchain nightly-2026-02-09 (rust-toolchain.toml); no macOS CI job exists
F1R3Gaze	main f6ee26a	@ "wallet added"; Rust 1.91, edition 2024; P4 release workflow producing a universal, notarised DMG
CampF1R3	main b46e16c	@ consumed only through F1R3Gaze, which pins branch gaze/map-remainders @ ea2d323; that revision is on GitHub, so a clean F1R3Gaze checkout builds
Publications	f1r3gaze/	architecture proposal v0.2; implementation specification v0.1 (work package K1 names installers in phase P4)
Publications	fuzzyware/	background only; nothing in the installer depends on the graded resolver, which F1R3Gaze does not yet ship

1.4 Conventions

MUST, MUST NOT, SHOULD, SHOULD NOT and MAY carry their RFC 2119 meanings. "The state directory" is `~/Library/Application Support/F1R3FLY/ign1t10n`. "The profile" is F1R3Gaze's profile directory, `~/Library/Application Support/F1R3Gaze` unless `F1R3GAZE_PROFILE` is set. `ign1t10n` names the product: the installer archive, the menu-bar application that manages the shard, its binary, its launch agent and its repository all carry the one name. Ports are written for the default allocation (§6.2).

2 Findings from the code review

These are the facts the design rests on. Each cites where it was established.

Finding 1 (The observer is not optional). Every endpoint that runs an exploratory deploy is served only by read-only nodes; a validator answers 400 `readonly_node_required`. The list (`docs/node/api-reference.md`) includes `POST /api/explore-deploy`, `POST /api/estimate-cost`, `GET /api/registry/{uri}` and `GET /api/balance/{address}`. F1R3Gaze's shard bridge uses exactly these: it resolves sites and reads registry values at observers (`gaze-shard/src/bridge.rs`, `graded`), quotes the cost of a deploy by `estimate_cost` on the first observer (line 276), subscribes to `/ws/events` on the first observer, and sends deploys to the validator. A standalone validator on its own therefore cannot serve the browser; the observer is what makes the local shard readable.

Finding 2 (The browser's defaults already match the port convention). `ShardConfig::default()` is validator `http://localhost:40403`, observers [`http://localhost:40453`], shard id `root`, quorum 2, phlo price 1. The node's documented port map puts the first node's HTTP API on 40403 and the observer's on 40453 (`docker/README.md`). With one observer the bridge returns the `node` rung and never consults the quorum (`bridge.rs`, the `obs.len() == 1` branch), so quorum 2 is harmless but misleading; the installer writes `quorum = 1`.

Finding 3 (Standalone plus observer is not a shipped topology). Standalone mode (`-s`, `standalone = true`) skips the wait for a first peer and the heartbeat's new-parents check, and still starts the transport and Kademlia servers (`node/src/rust/runtime/node_runtime.rs`, lines 135, 404, 574; `heartbeat_proposer.rs`, line 818). Nothing in the repository runs an observer against a standalone node: `docker/observer.yml` bootstraps from the multi-validator `shard.yml`. The join procedure itself (`docs/node/joining-a-network.md`) does not depend on the peer's mode, so the topology is expected to work, but it is unverified. See Obligation 1.

Finding 4 (The shipped development key is public). `docker/.env.example` (and the committed `docker/.env`) carry `STANDALONE_PRIVATE_KEY`, whose public key is the only bond in `docker/genesis/standalone-bonds.txt` and whose vault `1111AtahZ...` holds the whole standalone supply. Any install reusing it would give every reader of the repository the validator key and the funds.

Finding 5 (How a key can reach the node). `--validator-private-key` puts the key in the process's argument vector, which any local user can read with `ps`. `--validator-private-key-path` takes an encrypted PEM, decrypted with OpenSSL (`crypto/src/rust/signatures/secp256k1.rs`, `parse_pem_file`); the password comes from the environment variable `F1R3NODE_VALIDATOR_PASSWORD`, or else from an interactive prompt (`node/src/rust/effects/console_io.rs`). The environment of another user's process is not readable without privilege on macOS.

Finding 6 (Everything listens on all interfaces). `api-server.host` defaults to `0.0.0.0` (`defaults.conf`) and is configurable. The protocol transport binds `0.0.0.0` unconditionally (`comm/src/rust/transport/grpc_transport_receiver.rs`, line 600). On a laptop that would expose the internal gRPC API (which proposes blocks), the admin API and the peer protocol to whatever network the machine joins, and would trigger the application-firewall prompt.

Finding 7 (Native linking on macOS). The `crypto` crate depends on `openssl 0.10` without the vendored feature, so a macOS build links Homebrew's `libssl/libcrypto` dynamically; a binary built that way does not run on a Mac without Homebrew. LMDB is compiled from source through

heed; `aws-lc-sys`, `zstd-sys` and `bzip2-sys` link statically. `protoc` is a build-time requirement only. The repository’s workflows build Linux and OCI artifacts; none builds for macOS.

Finding 8 (F1R3Gaze already packages itself for macOS). `packaging/macos/package.sh` builds a universal (lipo) bundle, `io.f1r3fly.f1r3gaze`, minimum macOS 11, signs it with the hardened runtime and *empty* entitlements (there is no JavaScript, hence no JIT), wraps it in a signed DMG, notarises and staples it. Release secrets are `MACOS_CERT_P12`, `MACOS_CERT_PASSWORD`, `MACOS_SIGN_IDENTITY`, `APPLE_ID`, `APPLE_TEAM_ID`, `APPLE_APP_PASSWORD`. Wallet keys live in the Keychain under service F1R3Gaze (`os-keyring`, on in release builds).

Finding 9 (F1R3Gaze settings: last assignment wins). `settings.conf` is `key = value`; `Settings::parse` walks the lines in order and each recognised key overwrites the previous value (`gaze-shell/src/profile.rs`). If the file is absent at first load, F1R3Gaze writes a commented template. A block appended at the end of the file therefore governs the keys it sets without disturbing the rest.

Finding 10 (The browser’s wallet can be funded at genesis). F1R3Gaze wallet addresses are F1R3Cap addresses, base58 of a four-zero-byte prefix, a Keccak-derived hash and a Blake2b checksum (`gaze-wallet/src/address.rs`), the same form as the vault addresses in the node’s `wallets.txt`. The active wallet signs, and pays for, every deploy the browser makes. Balances and transfers go through Embers only; with `embers_api` unset the wallet panel reports that balances need Embers.

Finding 11 (Lifecycle facts). The node handles `SIGTERM` and `SIGINT` (`node_runtime.rs`, around line 908). A joined node restarted on the same data directory reconnects in about a second; its identity is the TLS certificate in the data directory, so a data directory must never be deleted while peers remember it (`joining-a-network.md`). Log output is governed by the logging block (`sink`, `format`, `rotation`, `retention`). The node reads `OPENAI_ENABLED` and `OLLAMA_ENABLED` from the environment. F1R3Gaze does not yet handle `f1r3://` links opened from other macOS applications (its README, “Not in this release”).

3 Architecture

3.1 Components

Component	Role
<code>F1R3Gaze.app</code>	The F1R3Gaze release bundle, byte for byte. The installer does not rebuild or re-sign it.
<code>ign1t10n.app</code>	New. Menu-bar application that provisions, starts, stops, resets and reports on the local shard. Contains <code>ign1t10n</code> (one Rust binary with three roles) and <code>f1r3node</code> (the F1R3Node-Rust node binary, universal).
launch agent	<code>io.f1r3fly.ign1t10n.supervisor</code> , registered from inside the <code>ign1t10n</code> bundle with <code>SMAppService</code> ; runs <code>ign1t10n supervise</code> , which owns both node processes.
validator	<code>f1r3node run --standalone</code> ; created the genesis; the only bonded validator; receives deploys.
observer	<code>f1r3node run</code> with no validator key; joined the validator by the last-finalised-state protocol; serves every read the browser makes.
state directory	Keys, genesis files, node configurations, both data directories, the manifest <code>shard.toml</code> , the control socket.

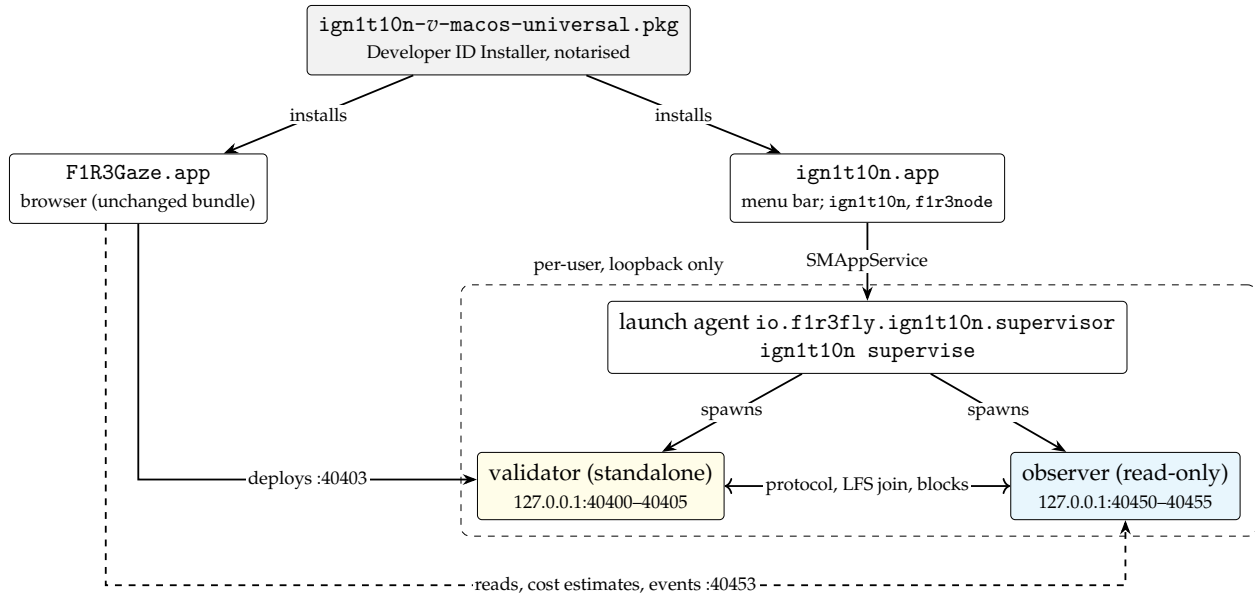


Figure 1: What the installer lays down and what runs afterwards.

3.2 Principles

Principle 1 (Per-user, unprivileged at run time). Administrator rights are used once, by Installer.app, to copy two bundles into /Applications. Everything after that runs as the logged-in user. Two users of one Mac each get their own shard.

Principle 2 (Loopback only). No socket of either node listens on a non-loopback address. The local shard is a private instrument, not a network service.

Principle 3 (Fresh secrets). Every secret is generated on the user’s machine at provisioning time. No key, password or seed is shipped in any artifact.

Principle 4 (One owner for the shard). A single supervisor starts, orders, monitors and stops both nodes. launchd keeps the supervisor alive; the supervisor keeps the nodes alive. Two independent launch agents were rejected because launchd has no dependency ordering and the observer’s bootstrap address depends on the validator having started.

Principle 5 (Touch only what was created). The installer and ign1t10n never delete or overwrite data they did not create. In particular they never remove a F1R3Gaze wallet, never edit F1R3Gaze settings outside their own managed block, and never use ~/ .rnode.

Principle 6 (Idempotent provisioning). Provisioning is a sequence of stages whose completion is recorded in shard.toml. Interrupted at any point (power loss, quit, crash) it resumes at the first incomplete stage.

4 The distribution artifact

Requirement 1 (Product archive). The release artifact MUST be a flat product archive, ign1t10n-v-macos-universal.pkg, built with productbuild from two component packages (io.f1r3fly.f1r3gaze.pkg, io.f1r3fly.ign1t10n.pkg), signed with a Developer ID

Installer certificate, notarised, and stapled. Each component package MUST install its bundle to /Applications with bundle-version checking on, so that an installed newer F1R3Gaze is not downgraded.

A DMG with two applications was considered and rejected: drag-and-drop cannot launch anything, and the brief asks that the install end with the shard and the browser running. A product archive delivers both bundles in one confirmation and has a post-install hook.

Requirement 2 (The post-install hook does one thing). The postinstall script of the component package `io.f1r3fly.ign1t10n.pkg` MUST do nothing but launch `ign1t10n` in the console user's session:

```
#!/bin/bash
user=$(stat -f %Su /dev/console)
[ "$user" = root ] || [ -z "$user" ] && exit 0      # no GUI session: first launch will
do it
uid=$(id -u "$user")
launchctl asuser "$uid" sudo -u "$user" \
    open -g -a /Applications/ign1t10n.app --args --first-run
exit 0
```

It MUST NOT create files in the user's home, register agents, or generate keys: all of that is provisioning, performed by `ign1t10n` as the user (§6). The script MUST exit 0 even if the launch fails, so that a headless install (MDM, `installer -pkg` over SSH) succeeds and provisioning happens at first launch.

Requirement 3 (Platforms). The artifact MUST run on Apple silicon and SHOULD run on Intel (Decision 1); every Mach-O in it is universal. The minimum macOS is 13.0, the first release with `SMAccessService` (Decision 2). `ign1t10n`'s `Info.plist` sets `LSMinimumSystemVersion` to 13.0; F1R3Gaze's own minimum (11.0) is unaffected, and the distribution definition sets `allowed-os-versions` to 13.0 and later.

Requirement 4 (No dynamic dependencies outside the system). For every Mach-O in the archive, `otool -L` MUST list only paths under `/usr/lib` and `/System/Library`. The release pipeline enforces this (§13); it is the check that catches Finding 7.

5 ign1t10n

5.1 Bundle layout

```
ign1t10n.app/Contents/
  Info.plist                io.f1r3fly.ign1t10n, LSUIElement = true, min 13.0
  MacOS/ign1t10n            menu bar (default), `supervise`, `ctl`
  Helpers/f1r3node          f1r3node-rust `node` v0.4.46, universal
  Library/LaunchAgents/io.f1r3fly.ign1t10n.supervisor.plist
  Resources/
    ign1t10n.icns, StatusTemplate.pdf  F1R3FLY.io brand kit (firefly bug)
    templates/validator.conf.in, observer.conf.in, gaze-settings.in
    compat.toml                      node versions whose data directories are compatible
    uninstall.sh
    _CodeSignature/ ...
```

5.2 One binary, three roles

ign1t10n is written in Rust (stable 1.91, matching F1R3Gaze), using objc2 bindings for AppKit (status item, first-run window), ServiceManagement (SMAppService) and Security (Keychain), k256 and pkcs8 for key generation and encryption, toml and serde for the manifest, and a blocking HTTP client for the node APIs. Rust keeps the supervisor, the key code and the configuration generator testable on Linux CI; only the thin UI and ServiceManagement layer is macOS-specific (Decision 3).

ign1t10n (no arguments)

The menu-bar application. On first run (`--first-run`, or `shard.toml` absent or incomplete) it opens the first-run window and drives provisioning. It never runs a node itself; it talks to the supervisor over the control socket.

ign1t10n supervise

The launch agent's program. Owns the two node processes (§7). Writes status to the control socket.

ign1t10n ctl *verb*

A command-line client of the control socket: `status [--json]`, `start`, `stop`, `restart`, `logs [validator|observer|ign1t10n]`, `env` (prints the endpoint URLs and the shard id for scripts), `reset`, `uninstall`. It is what the tests drive.

The control socket is `run/control.sock` in the state directory, a Unix-domain socket in a `0700` directory, carrying one JSON object per line. The supervisor MUST check the peer's user id (`getpeereid`) and refuse any other user.

5.3 The launch agent

```
<dict>
  <key>Label</key>           <string>io.f1r3fly.ign1t10n.supervisor</string>
  <key>BundleProgram</key>   <string>Contents/MacOS/ign1t10n</string>
  <key>ProgramArguments</key> <array><string>ign1t10n</string><string>supervise</string></array>
  <key>RunAtLoad</key>       <true/>
  <key>KeepAlive</key>       <dict><key>SuccessfulExit</key><false/></dict>
  <key>ProcessType</key>     <string>Standard</string>
  <key>ThrottleInterval</key> <integer>10</integer>
  <key>ExitTimeOut</key>     <integer>120</integer>
  <key>AssociatedBundleIdentifiers</key> <string>io.f1r3fly.ign1t10n</string>
</dict>
```

ProcessType is Standard, not Background: launchd throttles CPU and I/O of background jobs, which would slow block execution. KeepAlive restarts the supervisor after a crash but not after a deliberate stop, which exits 0. ExitTimeOut gives the supervisor time for the ordered shutdown of §7.4 before launchd escalates to SIGKILL.

5.4 The state directory

```

~/Library/Application Support/F1R3FLY/ign1t10n/      0700
shard.toml      the manifest (Appendix B)
keys/validator.pem      encrypted PKCS#8, 0600
genesis/bonds.txt, wallets.txt
conf/validator.conf, observer.conf
validator/      storage.data-dir of the validator (TLS identity, LMDB,
logs/)
observer/      storage.data-dir of the observer
run/control.sock, supervisor.pid
archive/<timestamp>/      previous shards kept on reset or incompatible upgrade
~/Library/Logs/F1R3FLY/ign1t10n/
ign1t10n.log      supervisor and UI; 10 MB x 5
validator.stdout.log, observer.stdout.log      panics and pre-logger output; 10 MB x 3

```

5.5 Keychain items

Service	Account	Content
io.f1r3fly.ign1t10n	validator-pem-password	32 random bytes, hex; the password of keys/validator.pem

The item is created by ign1t10n and its access control list is the application’s designated requirement, so the menu bar, the supervisor and ct1 (one binary) read it without a prompt and other programs cannot. The private key itself is never in the Keychain, never on a command line and never in a log; it exists in plaintext only in the node’s memory. F1R3Gaze’s wallet keys stay in F1R3Gaze’s own Keychain items; ign1t10n never reads them.

6 Provisioning

Provisioning runs in the menu-bar process, as the user, when shard.toml is absent or records an incomplete stage. Each stage below ends by recording itself in shard.toml’s [stages] table; a stage that finds its postcondition already true records itself and moves on.

6.1 S0: preflight

Checks: macOS $\geq$ 13; F1R3Gaze.app present with bundle identifier io.f1r3fly.f1r3gaze and a valid signature from the same team as ign1t10n; at least 10 GB free on the volume holding the home directory (warn below 20 GB); the state directory creatable. A failure stops provisioning with a specific message and a “Try again” button.

6.2 S1: ports

Each node needs six ports (protocol, gRPC external, gRPC internal, HTTP, discovery, admin). The default allocation is the node’s own convention, which is also F1R3Gaze’s default (Finding 2):

Node	protocol	gRPC ext	gRPC int	HTTP	discovery	admin
validator	40400	40401	40402	40403	40404	40405
observer	40450	40451	40452	40453	40454	40455

A block is usable if all six ports bind on 127.0.0.1. If a default block is taken (commonly by a Docker shard from `docker/shard.yml`, whose ports are published on all interfaces), try bases $b + 1000k$ for $k = 1, \dots, 9$, keeping the $+0 \dots +5$ offsets; the validator and observer are allocated independently. The chosen bases are recorded and reused on every start; they change only on explicit reallocation (§7.5).

6.3 S2: the paying wallet

The shard's genesis must fund the wallet F1R3Gaze will pay with (Finding 10). Using the interface of work package G1:

1. `f1r3gaze wallet active --porcelain`; if it prints an address, use it.
2. Otherwise `f1r3gaze wallet new "Local shard" --porcelain`, which creates a wallet, makes it active (it is the first), and prints its address.

Both run the F1R3Gaze binary inside F1R3Gaze.app with the user's profile. The address is recorded as `wallet.funded.ign1t10n` never sees the key.

6.4 S3: the validator key

Generate a secp256k1 key from the operating system's random source (k256, OsRng). Generate a 32-byte password, store it in the Keychain item above, and write the key as an ENCRYPTED PRIVATE KEY PEM: PKCS#8 with PBES2, PBKDF2-HMAC-SHA256 (600 000 iterations) and AES-256-CBC, which OpenSSL decrypts and Secp256k1::parse_pem_file accepts (Finding 5). Record the uncompressed public key (04..., 130 hex digits) and its F1R3Cap address. A unit test MUST round-trip a key generated this way through parse_pem_file from the pinned F1R3Node-Rust revision.

6.5 S4: genesis files

```
# genesis/bonds.txt: <validator public key> <stake>  
04a1...9e 1000  
# genesis/wallets.txt: <F1R3Cap address>, <balance in dust>      (8 decimals)  
<wallet.funded>, 10000000000000000 # 1 000 000 F1R3 for the browser's wallet  
<validator address>, 499000000000000000 # the rest of the dev supply: the local faucet
```

The amounts are Decision 4. The validator’s vault is funded so that the “Fund wallet” action (§8) can transfer to other F1R3Gaze wallets later without a new genesis.

6.6 S5: node configurations

Render `conf/validator.conf` and `conf/observer.conf` from the bundle's templates (Appendix A). Both set: the data directory and TLS paths inside the state directory; the six ports; `api-server.host = "127.0.0.1"`; the bind address of work package N1; the network id `ign1t10n-8 random hex digits` (fresh at each genesis, so that no other local shard can ever be mistaken for this one); the shard name `root`; logging to file, JSON, daily rotation, 7 files. The validator adds the standalone and genesis-ceremony blocks of `run-local/conf/standalone.conf`; the observer disables the heartbeat and has no ceremony block. Finality parameters (`fault-tolerance-threshold = 0.0`) are the same in both.

6.7 S6: register the agent

`SMAppService.agent(plistName: "io.f1r3fly.ign1t10n.supervisor.plist").register()`. If the status is `requiresApproval`, the first-run window explains that macOS asks for approval of background items, and offers a button that calls `SMAppService.openSystemSettingsLoginItems()`; provisioning waits, polling the status each second, and continues when the status becomes enabled.

6.8 S7: first start

The supervisor, now running, performs its normal start (§7.1) and additionally records, from the validator's `GET /api/status`, the node id in its address field, and the genesis block hash from `GET /api/last-finalized-block` at height 0. The first-run window shows each sub-step.

6.9 S8: configure F1R3Gaze

With work package G3, write `settings.d/50-ign1t10n.conf` in the profile. Until G3 lands, create `settings.conf` from F1R3Gaze's template if it is absent, and replace (or append) the block delimited by the two marker lines below; by Finding 9 it governs the keys it sets.

```
# >>> managed by ign1t10n: do not edit inside this block >>>
validator = http://127.0.0.1:40403
observers = http://127.0.0.1:40453
shard_id = root
quorum = 1
phlo_price = 1
# <<< managed by ign1t10n <<<
```

If the profile already sets `validator` or `observers` to something other than loopback, the first-run window asks before pointing F1R3Gaze at the local shard ("Use the local shard in F1R3Gaze", default yes). The choice is a menu toggle afterwards; turning it off removes the block.

6.10 S9: open the browser

`open -b io.f1r3fly.f1r3gaze --args gaze://newtab`. The browser is opened only once both nodes are ready, so that the first page a person sees can already reach the shard. The first-run window closes with a summary: shard id, the funded wallet and its balance, and where the logs are.

6.11 Time budget

Targets on an M-series Mac, measured by the acceptance suite: S0–S6 under 10 s of machine time (approval excluded); validator genesis to `/api/ready` under 60 s; observer join to `/api/ready` under 180 s (the join guide cites one to two minutes against a healthy shard); whole first run under five minutes.

7 Supervision

7.1 Start

1. Read `shard.toml`; refuse to start if the installed `f1r3node` version is incompatible with the recorded genesis version (§12.1).
2. Probe the recorded ports on `127.0.0.1`; on conflict go to §7.5.
3. Fetch the PEM password from the Keychain and spawn the validator (argument vector below) with environment `F1R3NODE_VALIDATOR_PASSWORD=...`, `OPENAI_ENABLED=false`, `OL-LAMA_ENABLED=false`, and nothing inherited beyond `HOME`, `PATH`, `TMPDIR`, `LANG`. Stdout and stderr go to `validator.stdout.log`.
4. Poll `GET /api/ready` each second until 200, or fail after 120 s (600 s on the genesis start).
5. Spawn the observer with the bootstrap address built from the recorded node id and the validator's protocol and discovery ports, with the same filtered environment minus the password. Poll its `/api/ready` likewise (300 s on the first join).
6. Report Running.

```
f1r3node run --standalone \  
  --config-file "$STATE/conf/validator.conf" \  
  --validator-private-key-path "$STATE/keys/validator.pem" \  
  --host 127.0.0.1 --no-upnp --allow-private-addresses \  
  --native-token-name F1R3CAP --native-token-symbol F1R3 --native-token-decimals 8  
  
f1r3node run \  
  --config-file "$STATE/conf/observer.conf" \  
  --bootstrap "rnode://$NODE_ID@127.0.0.1?protocol=40400&discovery=40404" \  
  --host 127.0.0.1 --no-upnp --allow-private-addresses --heartbeat-disabled \  
  --native-token-name F1R3CAP --native-token-symbol F1R3 --native-token-decimals 8
```

7.2 Health

Every 10 s the supervisor reads `GET /api/status` from both nodes and keeps `isReady`, `lastFinalizedBlockNumber` and the peer count. The shard is Degraded when either node is not ready, the observer has no peer, or the observer's last finalised block trails the validator's by more than 20 blocks for more than 60 s. The status machine is `Stopped` $\rightarrow$ `Starting(stage)` $\rightarrow$ `Running` $\rightleftharpoons$ `Degraded(reason)`, with `Failed(reason)` reachable from any state and left only by an explicit start.

7.3 Restarts

A node that exits unexpectedly is restarted with exponential back-off (1, 2, 4, ..., 60 s). A validator restart does not restart the observer; the observer reconnects on its own (Finding 11). Five unexpected exits of one node within five minutes put the shard in `Failed`, stop the other node, and post a user notification pointing at the log. The supervisor never deletes a data directory to recover.

7.4 Stop

On `SIGTERM` (logout, shutdown, `ctl stop`, agent unregistration): `SIGTERM` to the observer, wait up to 30 s; `SIGTERM` to the validator, wait up to 60 s; `SIGKILL` whatever remains; exit 0. The observer stops first so that it never sees the validator vanish mid-sync.

7.5 Port conflicts at start

If a recorded port is taken at start, the shard goes to Failed("port 40403 in use by <process>") (the holder named via lsof), and the menu offers "Choose new ports". Reallocating is safe: API ports are only in F1R3Gaze's managed block, which is rewritten; a new validator protocol port only changes the observer's bootstrap argument. No re-genesis is needed.

7.6 Sleep and wake

Nothing special is required: the nodes are processes under a launch agent, not an application, so App Nap does not apply. After wake the heartbeat proposes on its next interval. The supervisor SHOULD hold an IOPMAssertion of type PreventUserIdleSystemSleep only while provisioning, so a first genesis is not interrupted by idle sleep, and never otherwise.

8 User interface

8.1 Menu bar

A monochrome template image of the F1R3FLY.io firefly (from the brand portal, as F1R3Gaze's icons are) with a status dot: none for Running, amber for Starting and Degraded, red for Failed, hollow for Stopped. The menu:

Item	Action
<i>Local shard: Running</i>	status line; submenu shows block height, both nodes' readiness, observer lag, disk used
Open F1R3Gaze	<code>open -b io.f1r3fly.f1r3gaze</code>
Start / Stop shard	through the control socket
Fund a wallet...	transfer from the validator's vault to a chosen F1R3Gaze wallet address (work package H4)
Use the local shard in F1R3Gaze	toggles the managed settings block (§6.9)
Copy endpoints	validator and observer URLs, shard id
Show logs	reveals ~/Library/Logs/F1R3FLY/ign1t10n
Reset shard...	§12.2
Run shard at login	SMAppService register / unregister
Uninstall...	§12.3
Quit ign1t10n	quits the menu bar only; the shard keeps running

8.2 First-run window

One window, F1R3FLY.io brand on a white surface (the white-background logo variant; yellow and sky only as rules and progress, never as text), listing the stages S0–S9 with a spinner, check or error each, the elapsed time for the long ones, and a single primary button that changes from "Setting up..." to "Open F1R3Gaze". Errors name the stage, the cause and the log line.

9 Integration with F1R3Gaze

9.1 What the browser sees

A F1R3Gaze configured by S8 behaves exactly as against a remote shard with one observer: sites are resolved at the observer, answers carry the *node* rung, deploys are quoted with a cost estimated at the observer, signed by the active wallet, sent to the validator, and followed to finalisation through the observer's `/ws/events`. Pages that demand the *quorum* rung receive an error locally; that is correct, since one machine cannot provide distinct operators.

9.2 Required changes to F1R3Gaze

Work package 1 (G1: porcelain wallet commands). `f1r3gaze wallet active --porcelain` prints the active wallet's address and exits 0, or exits 3 if there is none; `f1r3gaze wallet new LABEL --porcelain` prints only the new address. Both work while a F1R3Gaze window is open (the keystore and `wallets.tsv` are safe for concurrent readers; `new` takes the profile's lock file). Estimate: 0.5 engineer-week.

Work package 2 (G2: balances without Embers). When `embers.api` is unset, the wallet panel and `wallet balance read GET /api/balance/{address}` from the first observer (the node's own read-only endpoint) instead of reporting that balances need Embers. Transfers still require Embers. Estimate: 0.5 engineer-week.

Work package 3 (G3: a settings drop-in directory). After `settings.conf`, F1R3Gaze parses each `settings.d/*.conf` in the profile in lexical order with the same last-assignment-wins rule. This lets `ign1t10n` own a file instead of a block inside the user's file. Estimate: 0.25 engineer-week.

Registering `f1r3://` and `f1r3h://` with Launch Services is left to F1R3Gaze's own roadmap (Finding 11); the installer adds nothing to F1R3Gaze's `Info.plist`.

10 Required changes to F1R3Node-Rust

Work package 4 (N1: loopback binding). Add `protocol-server.bind-address` and `peers-discovery.bind-address` (default `0.0.0.0`, preserving current behaviour) and use them where the transport receiver and the Kademlia server bind, replacing the literal in `grpc.transport.receiver.rs`. `ign1t10n` sets both, and `api-server.host`, to `127.0.0.1`. Test: with the three set, `lsof -iTCP -sTCP:LISTEN` shows no non-loopback socket for the process. Estimate: 1 engineer-week.

Work package 5 (N2: a self-contained macOS build). Add a vendored-openssl feature to `crypto` (forwarding to `openssl/vendored`) and a CI job on `macos-14` building `aarch64-apple-darwin` and `x86_64-apple-darwin` with it, running the unit tests on `arm64`, and failing if `otool -L` shows any library outside `/usr/lib` and `/System`. Estimate: 1 engineer-week.

Work package 6 (N3: standalone with an observer). Add `docker/standalone-observer.yml` (or an integration test) running a standalone validator and an observer bootstrapped from it, and a CI check that: the observer reaches Running; a deploy sent to the validator is reported finalised by the observer's `/api/deploy-finalization-status`; `/api/registry` and `/api/explore-deploy` answer at the observer at the new block; and the observer survives a restart of the validator. Fix whatever it exposes. This discharges Obligation 1. Estimate: 1–2 engineer-weeks.

Work package 7 (N4: version and compatibility metadata). `f1r3node --version` already exists through `clap`; add to it (and to `/api/status`) a *storage format* integer that changes whenever a release cannot open a data directory written by the previous one. `ign1t10n`’s `compat.toml` keys on it (§12.1). Estimate: 0.25 engineer-week.

11 Security model

Threat	Protection
Reuse of the public development key	Keys generated at S3; the release pipeline greps every artifact for the four development public keys of <code>docker/.env.example</code> and fails on a match.
Another user on the Mac	State directory 0700, PEM 0600; key never in <code>argv</code> ; PEM password only in the Keychain and in the validator’s environment; control socket checks the peer uid.
Hosts on the same network	Every listener on loopback (N1 and <code>api-server.host</code>); nothing registered for Bonjour.
Local web pages in other browsers	A stock browser can reach <code>127.0.0.1:40403</code> . The node’s HTTP API is unauthenticated, so a page elsewhere could submit a deploy it signed itself, paid by its own key, to the local shard. It cannot sign as the user’s wallet or the validator. Accepted for a development shard; Decision 6 proposes refusing cross-origin requests.
Pages inside F1R3Gaze	F1R3Gaze’s own model: capabilities, consent prompts quoting <code>phlo</code> , the system-name allow-list refusing <code>rho:rchain:deployerId</code> .
Tampered artifact	Developer ID signatures on every Mach-O and on the archive; notarisation; hardened runtime on <code>ign1t10n</code> and <code>f1r3node</code> with no entitlements; a GPG-signed SHA256SUMS beside the archive, as F1R3Gaze does.
Malware running as the user	Not defended. It can read the state directory; the hardened runtime at least prevents attaching a debugger to the node to read the decrypted key.

12 Upgrade, reset and uninstall

12.1 Upgrade

A newer archive replaces both bundles in place. The supervisor, restarted by `launchd` because its executable changed, compares the new `f1r3node`’s storage format (N4) with the one recorded at genesis:

- *compatible*: restart both nodes on their existing data;
- *incompatible*: do not start; the menu explains and offers “Start a new shard”, which archives the current shard under `archive/` (§12.2) and re-provisions from S3. The wallet is kept and funded again.

A new node binary is never started against a data directory it cannot read. F1R3Gaze upgrades independently of the shard.

12.2 Reset

“Reset shard...” stops the shard, moves `validator/`, `observer/`, `genesis/`, `conf/` and `keys/` into `archive/<timestamp>/` (deleting archives beyond the newest two, after confirmation, when free space is short), deletes the Keychain item, and re-runs provisioning from S1. The dialog says that every deploy, registry entry and balance on the local shard will be gone, that F1R3Gaze wallets are untouched, and that the active wallet will be funded again.

12.3 Uninstall

“Uninstall...” (also `Resources/uninstall.sh` and `ctl uninstall`): stop the shard; unregister the agent; remove the state directory (after a second confirmation, with an option to keep archive/), the logs and the Keychain item; remove F1R3Gaze’s managed block or drop-in file; move `ign1t10n.app` to the Trash; `pkgutil --forget io.f1r3fly.ign1t10n.pkg`. F1R3Gaze, its profile and its wallets are left alone; the dialog says so and names the F1R3Gaze uninstall route (drag to Trash) for anyone who wants it gone too.

13 Build and release pipeline

13.1 Where it lives

A new repository, `F1R3FLY-io/ign1t10n`, holding the `ign1t10n` crate, the templates, `packaging/-macos/`, and `versions.toml`, which pins the F1R3Node-Rust revision and the F1R3Gaze release (Decision 7). Pushing a tag `vX.Y.Z` runs the release workflow.

13.2 Jobs

Job	Runner	Steps
node	macos-14	check out F1R3Node-Rust at the pinned revision; toolchain from its <code>rust-toolchain.toml</code> ; <code>brew install protobuf</code> ; <code>build -p node --release --features crypto/vendored-openssl</code> for both targets; <code>lipo</code> to <code>f1r3node</code> ; <code>otool -L gate</code>
gaze	ubuntu	download <code>F1R3Gaze-v-macos-universal.dmg</code> , <code>SHA256SUMS</code> and its signature from the pinned F1R3Gaze release; verify the GPG signature and checksum
ign1t10n	macos-14	build <code>ign1t10n</code> for both targets; unit tests (including the PEM round-trip against the pinned F1R3Node-Rust); <code>lipo</code>
assemble	macos-14	mount the F1R3Gaze DMG and check <code>codesign --verify --deep --strict</code> and <code>spctl</code> ; assemble <code>ign1t10n.app</code> ; sign inside-out; <code>pkgbuild</code> both components; <code>productbuild --distribution</code> ; sign with the installer certificate; notarise; <code>staple</code> ; <code>pkgutil --check-signature</code> ; <code>spctl -a -t install</code>
smoke	macos-14	<code>sudo installer -pkg ... -target /</code> ; run provisioning headless (<code>ign1t10n ctl provision --non-interactive</code> , which skips S6 approval by launching the supervisor directly); the acceptance checks A2–A6 of §14
publish	ubuntu	<code>SHA256SUMS</code> over the archive, GPG signature; draft GitHub release

13.3 Signing order

```
cs() { codesign --force --timestamp --options runtime \
    --entitlements packaging/macos/entitlements.plist --sign "$APP_ID" "$@"; }
cs ign1t10n.app/Contents/Helpers/f1r3node
cs ign1t10n.app/Contents/MacOS/ign1t10n
cs ign1t10n.app                               # seals the bundle, incl. the agent
    plist
pkgbuild --component ign1t10n.app --install-location /Applications \
```

```

    --scripts packaging/macos/scripts --identifier io.f1r3fly.ign1t10n.pkg ...
pkgbuild --component F1R3Gaze.app --install-location /Applications \
    --identifier io.f1r3fly.f1r3gaze.pkg ... # the release bundle, not re-
    signed
productbuild --distribution Distribution.xml --package-path . unsigned.pkg
productsign --sign "$INSTALLER_ID" --timestamp unsigned.pkg ign1t10n-$V-macos-
    universal.pkg
xcrun notarytool submit ... --wait && xcrun stapler staple ign1t10n-$V-macos-universal.
    pkg

```

Secrets: those of F1R3Gaze (Finding 8), plus `MACOS_INSTALLER_CERT_P12`, `MACOS_INSTALLER_CERT_PASSWORD` and `MACOS_INSTALLER_SIGN_IDENTITY` for the *Developer ID Installer* certificate, which is distinct from the application certificate; and `GPG_PRIVATE_KEY`, `GPG_PASSPHRASE`. As in F1R3Gaze, a build without signing secrets still produces an unsigned archive, with a warning, for internal testing. The entitlements file is empty: neither binary needs JIT, unsigned executable memory or library-validation exceptions.

14 Testing and acceptance

14.1 Acceptance criteria

A1 Clean install.

On a fresh user account of a supported Mac, opening the archive and completing `Installer.app` (plus one background-item approval) ends with F1R3Gaze open and `ctl` status reporting `Running`, within five minutes.

A2 Round trip through the browser.

`f1r3gaze --headless` on a test page that deploys a registry insertion (consent granted with `--allow`) is signed by the funded wallet, reported finalised, and the inserted value is then read back at the observer at the *node* rung.

A3 Loopback only.

`lsof -nP -iTCP -sTCP:LISTEN` lists no socket of either `f1r3node` on an address other than `127.0.0.1`.

A4 No foreign libraries.

`otool -L` on every Mach-O lists only `/usr/lib` and `/System` paths.

A5 No shipped secrets.

The validator's public key differs from every key in `docker/.env.example`; the PEM is 0600; `ps -Eww` run by another user does not reveal the password or key.

A6 Persistence.

After a reboot and login, the shard returns to `Running` with the same genesis hash and a block height no lower than before.

A7 Port conflict.

With `docker/shard.yml` running, provisioning chooses the next block for the validator and observer and F1R3Gaze's settings name them; A2 passes.

A8 Crash recovery.

`kill -9` of either node leads back to `Running` without user action; six rapid kills lead to `Failed` and a notification.

A9 Upgrade.

Installing $v+1$ over v with a compatible storage format keeps the chain; with an incompatible one, the old shard is archived only after the user agrees, and the wallet is funded at the new genesis.

A10 Uninstall.

After uninstall, no file under the state directory or the log directory, no Keychain item of service `io.f1r3fly.ign1t10n`, and no registered agent remain; F1R3Gaze and its wallets are untouched.

14.2 Matrix

macOS 13, 14, 15, 26 and the current release, on Apple silicon; macOS 13 and 15 on Intel while x86.64 is shipped. A1, A6 and A10 run on physical or virtualised machines before each release; the others run in CI on every tag.

15 Work packages and schedule

WP	Deliverable	Est.
N1	node: configurable bind address for transport and discovery	1
N2	node: vendored OpenSSL feature; macOS CI build with <code>otool</code> gate	1
N3	node: standalone-plus-observer topology test, and fixes	1.5
N4	node: storage-format number in <code>--version</code> and <code>/api/status</code>	0.25
G1–G3	F1R3Gaze: porcelain wallet commands; balances from the node; <code>settings.d</code>	1.25
H1	ign1t10n: manifest, provisioning stages, configuration rendering, key generation and PEM, Keychain	2
H2	ign1t10n supervise: process management, health, restarts, shutdown, control socket, <code>ctl</code>	2
H3	menu bar, first-run window, <code>SMAccessService</code> , notifications, brand assets	2
H4	reset, upgrade compatibility, uninstall, “Fund a wallet”	1
R1	release workflow: packaging, signing, notarisation, smoke job	1.5
T1	acceptance suite and the pre-release matrix	1.5
Total		15

N3 is on the critical path: if the standalone topology needs node changes, the fallback in Obligation 1 costs about one more week. With two engineers (one on the node and pipeline, one on ign1t10n) the work fits in eight calendar weeks. Nothing here waits on F1R3Gaze phase P4 beyond the release artifact that F1R3Gaze already produces.

16 Obligations and decisions

Obligation 1 (The topology works). Before H2 is merged, N3 must show an observer joining and following a standalone validator. If standalone mode cannot serve the join, the fallback is to run the validator without `--standalone` but with the standalone configuration’s ceremony block (it then waits for its first peer, which the observer provides), and to make the heartbeat’s new-parents check conditional on the size of the active validator set rather than on the standalone flag.

Obligation 2 (PEM compatibility). The PBES2 parameters of S3 must be accepted by the OpenSSL that N2 vendors. The round-trip test in H1 is the proof and runs in CI against the pinned F1R3Node-Rust.

Obligation 3 (Finalisation latency). The shipped standalone configuration proposes from the heartbeat with a 5s check interval. A2 records the time from deploy to finalisation seen at the observer; if the median exceeds 10s the validator template’s heartbeat interval is revisited (Decision 5).

Open decision 1 (Intel). Ship universal binaries, as F1R3Gaze does, or Apple silicon only, which halves the node build time and removes an untested platform from the matrix.

Open decision 2 (Minimum macOS). 13.0, for SMapService. Supporting 11 and 12 would require writing a plist into ~/Library/LaunchAgents and using launchctl bootstrap, a second code path for two releases that are out of security support.

Open decision 3 (Implementation language of the UI layer). Rust throughout (proposed), or a Swift/SwiftUI menu bar calling the Rust supervisor, which is less code for the window and SMapService but adds a second toolchain.

Open decision 4 (Genesis amounts). One million F1R3 to the browser’s wallet and the rest of the standalone supply to the validator’s vault as a faucet; or fund every wallet F1R3Gaze lists at provisioning.

Open decision 5 (Heartbeat interval). Keep the repository’s standalone values (5s check, 10s maximum age of the last finalised block) or shorten them for a more responsive local shard.

Open decision 6 (Cross-origin requests). Whether ign1t10n’s node configuration should (after a small node change) refuse HTTP requests carrying an Origin header other than none, so that pages in other browsers cannot use the local shard’s API.

Open decision 7 (Repository). A new ign1t10n repository (proposed), or packaging/macos-local/ in the F1R3Gaze repository, which would couple the shard manager’s releases to the browser’s.

Open decision 8 (Embers). Whether a later version bundles an Embers instance so that transfers, not just balances, work against the local shard.

A Configuration templates

conf/validator.conf

```
# Rendered by ign1t10n. The node loads its built-in defaults.conf first.
standalone = true
dev-mode    = true

protocol-server {
  network-id = "ign1t10n-{{NETWORK_SUFFIX}}"
  port = {{V_BASE}}                        # 40400
  bind-address = "127.0.0.1"                # N1
  allow-private-addresses = true
  no-upnp = true
```

```

}
peers-discovery { port = {{V_BASE+4}}, bind-address = "127.0.0.1" }
api-server {
  host = "127.0.0.1"
  port-grpc-external = {{V_BASE+1}}
  port-grpc-internal = {{V_BASE+2}}
  port-http          = {{V_BASE+3}}
  port-admin-http    = {{V_BASE+5}}
}
storage { data-dir = "{{STATE}}/validator" }
tls {
  certificate-path = "{{STATE}}/validator/node.certificate.pem"
  key-path        = "{{STATE}}/validator/node.key.pem"
}
casper {
  shard-name = "root"
  fault-tolerance-threshold = 0.0
  synchrony-constraint-threshold = 0.0
  casper-loop-interval = 5 seconds
  requested-blocks-timeout = 60 seconds
  fork-choice-stale-threshold = 30 seconds
  fork-choice-check-if-stale-interval = 1 minutes
  max-number-of-parents = 100
  enable-mergeable-channel-gc = true
  heartbeat { enabled = true, check-interval = 5 seconds, max-lfb-age = 10 seconds }
  genesis-block-data {
    genesis-data-dir = "{{STATE}}/genesis"
    bonds-file      = "{{STATE}}/genesis/bonds.txt"
    wallets-file    = "{{STATE}}/genesis/wallets.txt"
    number-of-active-validators = 1
    pos-multi-sig-quorum = 1
  }
  genesis-ceremony {
    required-signatures = 0
    approve-interval = 1 seconds
    approve-duration = 1 seconds
    autogen-shard-size = 1
    ceremony-master-mode = true
  }
}
logging {
  format = "json"
  sink   = "file"
  file { rotation = "daily", retention = 7 }
}
metrics { prometheus = true }

```

conf/observer.conf

```

protocol-server {
  network-id = "ign1t10n-{{NETWORK_SUFFIX}}"      # identical to the validator's
  port = {{O_BASE}}                                # 40450

```

```

    bind-address = "127.0.0.1"
    allow-private-addresses = true
    no-upnp = true
}
peers-discovery { port = {{0_BASE+4}}, bind-address = "127.0.0.1" }
api-server {
    host = "127.0.0.1"
    port-grpc-external = {{0_BASE+1}}
    port-grpc-internal = {{0_BASE+2}}
    port-http          = {{0_BASE+3}}
    port-admin-http    = {{0_BASE+5}}
}
storage { data-dir = "{{STATE}}/observer" }
tls {
    certificate-path = "{{STATE}}/observer/node.certificate.pem"
    key-path        = "{{STATE}}/observer/node.key.pem"
}
casper {
    shard-name = "root"
    fault-tolerance-threshold = 0.0
    synchrony-constraint-threshold = 0.0
    max-number-of-parents = 100
    enable-mergeable-channel-gc = true
    heartbeat { enabled = false }
    genesis-block-data {
        genesis-data-dir = "{{STATE}}/genesis"
        bonds-file       = "{{STATE}}/genesis/bonds.txt"
        wallets-file     = "{{STATE}}/genesis/wallets.txt"
    }
}
logging { format = "json", sink = "file", file { rotation = "daily", retention = 7 } }
metrics { prometheus = true }

```

{{STATE}} contains a space (Application Support); HOCON accepts it inside quotes, and every path above is quoted. The bind-address keys are introduced by N1; until N1 lands the renderer omits them.

B The manifest

```

# ~/Library/Application Support/F1R3FLY/ign1t10n/shard.toml
schema = 1

[shard]
id = "root"
network_id = "ign1t10n-3f9c1a07"
genesis_hash = "a41c...e2"           # block 0, recorded at S7
genesis_node_version = "0.4.46"
storage_format = 7                   # from N4
created = 2026-10-14T09:12:44Z

[validator]
public_key = "04a1...9e"

```

```

address = "1111...Qk"
node_id = "5b0e...c1"
ports = { base = 40400 }

[observer]
ports = { base = 40450 }

[wallet]
funded = "1111...Zr"
gaze_integration = true

[stages]
preflight = true
ports = true
wallet = true
key = true
genesis = true
configs = true
agent = true
first_start = true
gaze = true
opened = true

```

from /api/status .address at S7

the Gaze wallet funded at genesis
the managed settings block is present

provisioning progress, S0..S9